Team ChiK-fiL-a: Keyon Jazayeri, Colin Kwon, Lex Truong
Professor Berg
CS 178: Machine Learning and Data-Mining
12 Dec 25

1. Problem Statement

Interested in the cross-section between machine learning and biology, we chose to implement a classifier that predicts whether an allele variant is "benign" or "pathogenic." This problem provides an opportunity for us to showcase how machine learning methods can support biological analysis, specifically in the field of genomics.

2. Dataset

2.1 Dataset Curation and Feature Selection (Colin)

We constructed a high-quality dataset for missense variant pathogenicity prediction by integrating clinically annotated variants from UniProt's Humsavar database with rich functional genomic features from dbNSFP 5.3a.

The curation process involved three key steps:

1. Label Standardization: The raw Humsavar file was first parsed from its fixed-width text format into a structured CSV. The heterogeneous clinical categories were then standardized into two unambiguous classes: Pathogenic and Benign. Variants labeled with uncertain significance were deliberately excluded to ensure the reliability of the clinical labels.
2. Feature Augmentation: We merged these labeled variants with dbNSFP annotations. To handle the large volume of data efficiently and conserve memory, we employed a stream-based approach across all chromosome-level dbNSFP files, extracting matching records using rsIDs. This produced a combined dataset containing widely used functional prediction scores, including SIFT, PolyPhen-2 HDIV, CADD, and REVEL.
3. Quality Control : A cleaning stage removed any variants lacking essential genomic coordinates (chromosome, position, reference allele, alternate allele, or rsID). We then finally compiled a dataset with rows that had no missing columns.

The complete process yielded 10941 high-quality variants suitable for machine-learning analysis.

Our feature selection strategy followed a domain-informed approach rather than relying purely on automated statistical methods. We specifically chose SIFT, PolyPhen-2, CADD, and REVEL because they are among the most extensively validated and biologically interpretable predictors in human variant analysis.

Each selected predictor captures a distinct mechanistic dimension of variant impact:

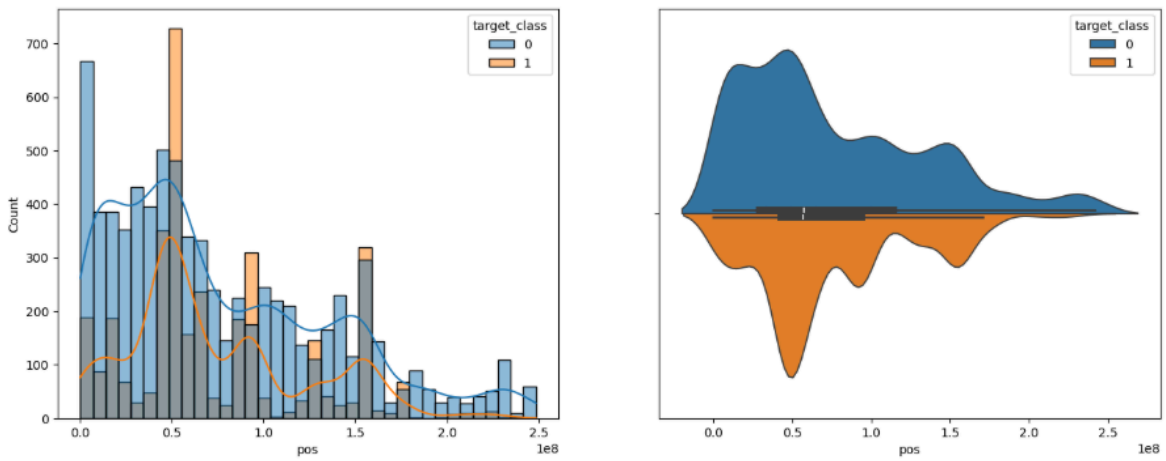
- SIFT: Predicts impact based on evolutionary conservation.
- PolyPhen-2 HDIV: Assesses the impact of amino acid substitution on protein structure and function.
- CADD: Provides a comprehensive score of genome-wide integrated deleteriousness.
- REVEL: Offers an ensemble consensus score trained directly on known pathogenic and benign variants.

Other similar studies have demonstrated that combining such complementary predictors produces more robust and clinically meaningful estimates of pathogenicity than single predictors alone. By selecting features with strong mechanistic grounding and proven clinical relevance, we ensure that the model benefits from high-quality, interpretable signals while reducing the risk of overfitting or including features with unclear functional significance.

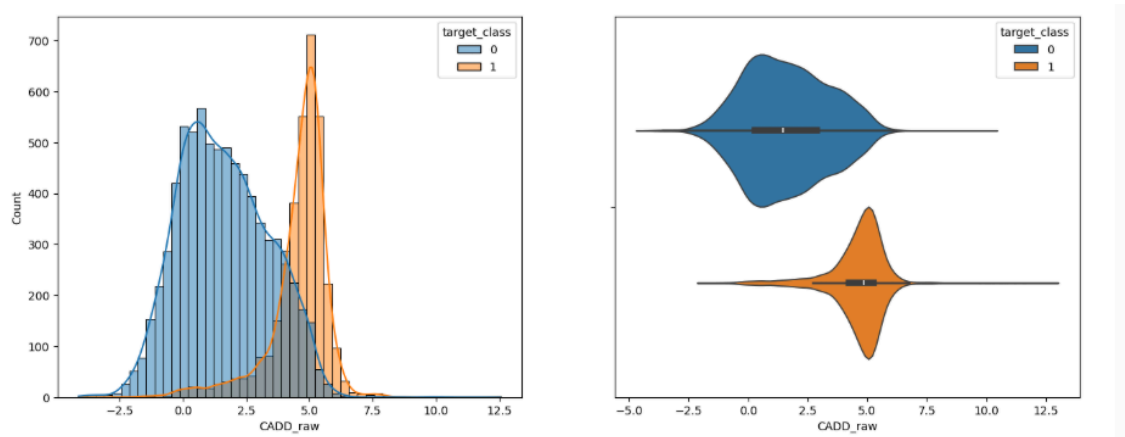
The final curated dataset consists of genomic coordinates, the four functional prediction scores, optional protein metadata, and a binary pathogenicity label. This resource provides a clean and biologically relevant foundation for downstream model development, enabling rigorous classifier evaluation while maintaining interpretability in a clinically significant domain.

2.2 Dataset Exploration and Analysis (Keyon)
Before training any models, we did some data exploration to get a sense of our features and identify which features may be strongly/loosely correlated with the target feature. The first thing we modeled was the distribution of the target class, and we found that ~⅔ of our data was benign and the remaining ⅓ was pathogenic. We had to keep this imbalance in mind when training our models.

Then I wanted to analyze our features and see how insightful they were regarding the target class. I decided to do one feature at a time and analyze its distribution when conditioned on the target class. I figured that differences in this distribution would mean there's a strong correlation between the two.



The figure above shows an example. This shows one feature, “pos,” and its distribution when Y=1 and Y=0. I looked at the mean, the skew, the peaks, etc. to see if there was any meaningful information gain. This particular distribution is very similar for both classes, which told me there might not be much value (at least, not by itself. It’s possible that when conditioned on another feature, it could show stronger correlation. This was not tested, and is something we could do if we had more time).



Some distributions looked more like the one above. Here we can clearly see that there is a difference in distribution per class, which told me that this feature is highly important. This was included in our model training.

From this exploration we drew conclusions on which features were most important, and which were least important. There was much more analysis done–refer to the notebook to go more in depth–but this is a high level overview of what we looked for.

3. Models

3.1. Baseline: Majority Class Classifier (Lex)
Since our dataset had such an imbalanced target feature distribution of ~⅔, we decided to use a majority class classifier as our baseline model. This classifier, which predicts "benign" every time, yielded 68% accuracy. Naturally, our objective was to explore models that substantially outperformed this 68% baseline.

3.2 Naive Bayes (Keyon)
Looking at the data exploration graphs and their conditional distributions inspired me to explore naive bayes, since naive bayes relies on conditional probabilities for its prediction. But, there were a couple of problems.

First, we had to make sure our assumption of conditional independence held. For the most part this was true. While we didn't test the independence mathematically, we knew intuitively (based on the actual features definition) that there was no relationship. But, there were a couple of features that had explicit ties to each other that had to be dealt with.

These features were tied together because one was the raw sample, and the other was binned (e.g. SIFT_score and SIFT_pred). Thankfully it was pretty straightforward to fix this. I just dropped one of the related features and kept the one that contained more information, the raw version. Problem solved!

The next challenge was that we had a mix of data and distribution. Naive Bayes has to calculate the likelihood of our features given the target class, which means we need a model of the feature's distribution. For categorical variables this is easy, it's just an empirical likelihood. But for our continuous features this was much more challenging because some of them had very skewed or otherwise abnormal distributions that were hard to capture.

. The question was twofold:

1. How do we model these weird distributions?
2. How do we store and represent each model's unique distribution? (ie we can't just use the built in sklearn Gaussian Naive Bayes classifier)

To solve these problems I tried out two solutions:

1. Bin the continuous variables into discrete buckets.
2. Use Kernel Density Estimations to model the distribution of our continuous features.

Starting with the first solution, binning continuous variables. This solved our problem because it allowed us to treat our continuous variables as categorical, discrete values. I also figured we

wouldn't lose a lot of information because a lot of our continuous distributions were in a very small range (like 0 - 1) and oftentimes they were somewhat smooth, so using the bins as an approximation felt fair (this is another way that EDA came in handy!) With that, I trained a classifier that got 89% accuracy and 0.83 F1.

The KDE naive bayes was inspired by the KDE estimates that the Seaborn library does in histograms and violin plots. They seemed like great estimates of the distribution, so I used scikit learn's built in KDE fitter to create a custom naive bayes class that modeled each continuous distribution with a KDE. Unfortunately, after all this effort, we saw marginal improvements to our model. 90% accuracy and 0.85 F1.

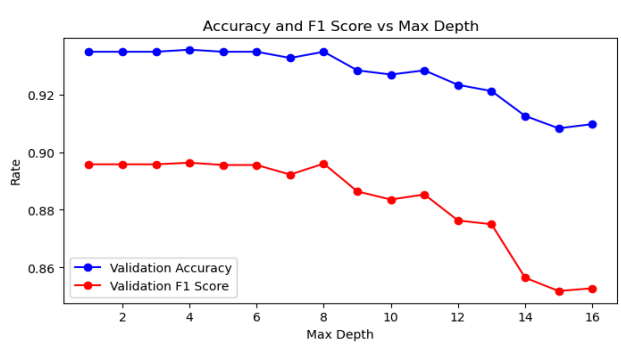
I also want to note that I did tune the bandwidth of our KDE naive bayes using sci-kit learn's gridsearch function. I found, surprisingly, that lower bandwidth did much better. I expected this to overfit to the training data's distribution but I was wrong!

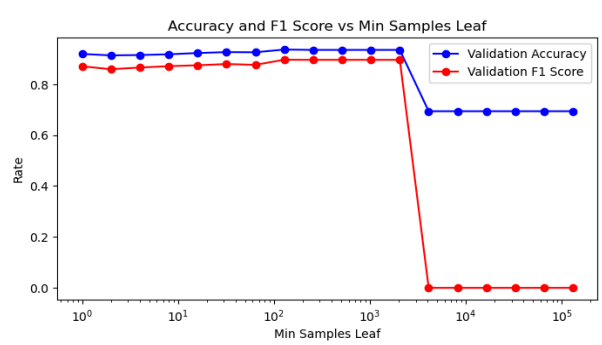
3.3 Decision Tree (Lex)

After looking at the first graph from Keyon's data exploration, one might conclude that the "pos" feature has little to no effect on the target class. I sought to challenge this assumption by using scikitlearn's decision trees, which can capture nonlinear relationships and feature interactions. On its own, "pos" may have little predictive power. But in conjunction with other features such as chromosome, "pos" could enhance overall model performance. Similarly, the "ref" and "alt" features may have little significance when isolated, but together they represent an allele mutation with predictive value.

The default model returned 90% accuracy and 0.88 F1. I then trained on criterion, max depth, and minimum leaf samples. Lower values for max depth or min leaf samples would increase the likelihood of overfitting, so I made sure to evaluate higher values as well.

I also experimented with weighing each class differently since our data distribution was skewed, but using no weights surprisingly had the best results.





Controlling complexity with minimum leaf samples yielded slightly better accuracy and F1 than with max depth. The best value for min leaf samples was 128. I also found that the best criterion was "entropy." Combined, our final decision tree model yielded 92% accuracy and 0.87 F1. This was a significant improvement above our baseline classifier and the default decision tree, but still concerningly low considering the problem we are addressing.

3.4 Random Forest (Lex)

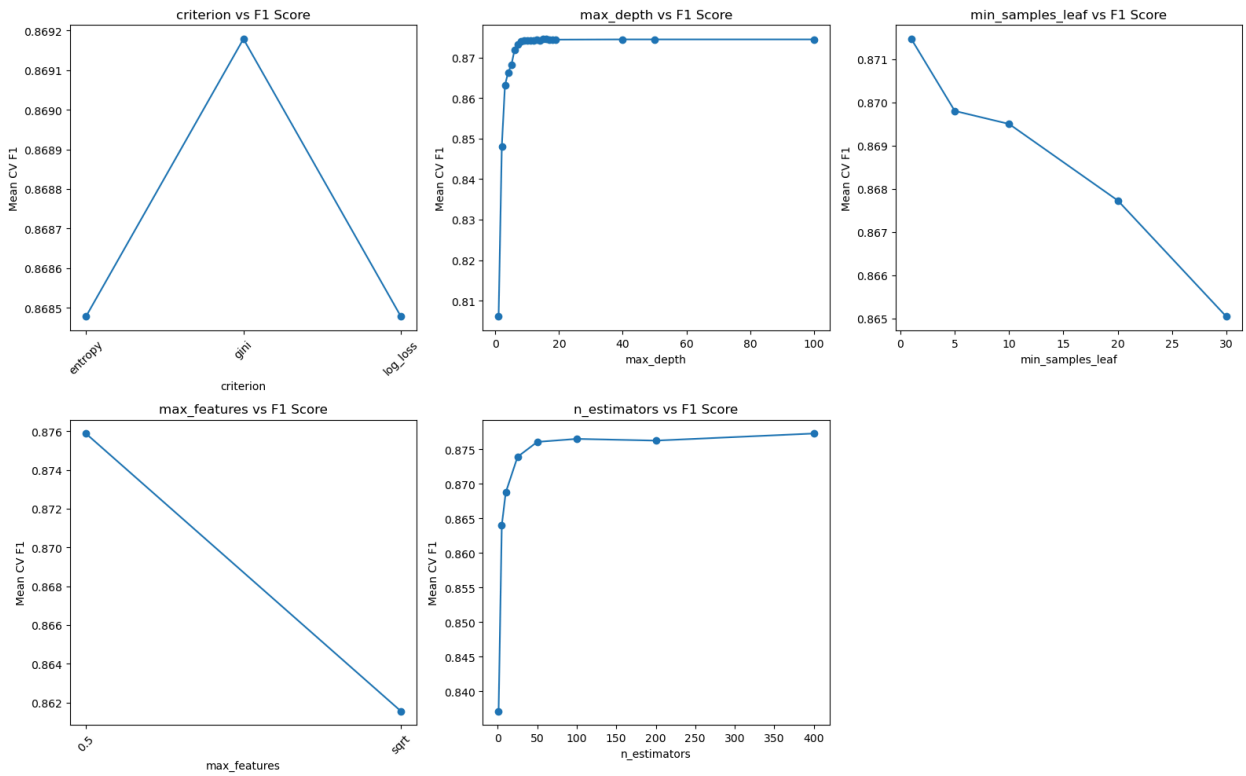
Seeking to extend the success from our decision tree model, I decided to train random forests since they similarly capture relationships between dataset features. And since they are composed of several decision trees, they naturally counteract overfitting by reducing the significance of noisy data. By averaging the results of its individual predictors, forests reduce variance with little trade-off in bias. As a result, I thought random forests had much potential.

The default scikitlearn random forest yielded 92% accuracy and 0.91 F1, already better than our decision tree model after tuning.

I then used GridSearchCV to vary values over an exhaustive grid of hyperparameters–criterion, max depth, min leaf samples, max features, and n estimators. After 37 minutes of runtime, the grid search asserted the best model had these parameters:

```
Best Model Parameters: {'criterion': 'gini', 'max_depth': 16,
'max_features': 0.5, 'min_samples_leaf': 1, 'n_estimators': 100}
```

But disappointingly, my search increased model performance minimally, with 93% accuracy and 0.92 F1



Additionally, plotting each grid parameter against F1 shows that the number of estimators had the greatest impact on model performance.

3.5 XGBoost (Keyon)

Finally, given our success with tree-based models, we decided to try a variation of gradient boosting called XGBoost. We did not learn this in class, so I spent some time learning about the model to make sure I understood what was happening.

To summarize what I learned, XGBoost is very similar to gradient boost in that it builds small trees to repeatedly model the loss of a very naive initial tree. XGBoost is different in a couple of key ways:

1. Its trees don't literally output the output, but something intermediary that is used to calculate the final prediction
2. It has built in regularization
3. It runs in parallel which makes the greedy decision tree algorithm much more efficient.

I first trained XGboost with no parameter tuning, just using the default classifier. We got a nice 94% accuracy, 90.9% F1 right out the box.

This was great, but the precision/recall of our pathogenic class (which has less data) was significantly lower than our benign class. Thankfully, XGBoost takes in a parameter to control the weights applied to each class. I weighted the class according to the ratio of benign/pathogenic in our training data, and found that we got 94.29% accuracy and 0.91 F1.

This looks similar on the surface, but when I looked at the per-class precision/recall I saw that we did much better on the pathogenic class.

	precision	recall	f1-score
0	0.94	0.95	0.95
1	0.89	0.87	0.88

(even weights)

	precision	recall	f1-score
0	0.96	0.96	0.96
1	0.90	0.92	0.91

(weighed according to ratio)

Our recall went up by 5%, which is great! In applications of a predictor like this one, we care more about recall (we want to identify every pathogenic cell, even if that means more false positives).

While we did not have time to fine tune every parameter for XGBoost, of which there are many, the default settings do a great job at finding the balance between over and underfitting.

4. Conclusion

Each of us built on one another's insights to advance our model performance and, more importantly, to follow our curiosities. Across Naive Bayes, decision trees, random forests, and XGBoost, we consistently saw that leveraging curated biological predictors such as SIFT, PolyPhen-2, CADD, and REVEL allowed our models to significantly surpass baseline performance. Our iterative research process demonstrates the importance of integrating the work of others, such as biological feature analysis, in genomic interpretation.

We offer XGBoost as our best model with 94% accuracy, 0.92 F1, and 0.92 recall. While we are pleased with our results, we feel our model's performance is limited by the quality of our data. Larger variant sets and additional features such as more functional genomic scores; protein structure or location; and allele sequence context would likely enable more predictive models.